

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Курсовая работа

по дисциплине: «Машинное обучение»

на тему: «Атаки в системах машинного обучения»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

Доцент, д.т.н. _____ Сараев П.В.

Липецк, 2025

Содержание

1. Цель работы 3
2. Задание 3
3. Исходный датасет 4
4. Программа для обучения модели 5
5. Анализ уязвимостей модели 7
6. Атакующие алгоритмы 9
7. Выводы 20
8. Список источников 21

1. Цель работы

Реализовать и проанализировать модель машинного обучения, решающую задачу классификации, а также исследовать её устойчивость к различным видам атак.

2. Задание

Выбранная тематика: 8. Атаки в системах машинного обучения.

1. Подобрать интересующее соревнование и извлечь из него датасет

2. Написать программу, которая обучит модель на выбранном датасете

3. Провести анализ уязвимости полученной модели

4. Провести атаки на модель.

5. Сравнить атакованные и нетронутые результаты.

6. Предложить усиление слабых мест модели.

3. Исходный датасет

В качестве задания, был выбран датасет, использующийся в соревновании «Titanic - Machine Learning from Disaster» с сервиса ***kaggle.com***, тематикой которого, как видно из названия, является крушение одноименного судна.

Датасет включает в себя всех пассажиров "Титаника", находящихся на нем в момент крушения. Задача соревнования: обучить модель таким образом, чтобы она могла определить по входным данным пассажира спасется он или нет в результате крушения.

1. `survival` — факт выживания (0 — нет, 1 — да)
2. `pclass` — класс билета (1 — первый, 2 — второй, 3 — третий)
3. `sex` — пол пассажира
4. `age` — возраст
5. `sibsp` — количество супругов и детей на борту
6. `parch` — количество родителей и детей на борту
7. `ticket` — номер билета
8. `fare` — стоимость проезда
9. `cabin` — номер каюты
10. `embarked` — порт посадки (C — Шербур, Q — Квинстаун, S — Саутгемптон)

Пояснение к переменным:

pclass: показатель социально-экономического статуса

- 1-й - Высший
- 2-й - Средний
- 3-й - Нижний

age: возраст указывается дробно, если он меньше 1. sibsp: набор данных определеяет семейные отношения следующим образом: Дети - брат, сестра, сводный брат, сводная сестра. Супруг - муж, жена

parch: Набор данных определяет семейные отношения следующим образом: Родитель - мать, отец Ребенок - дочь, сын, падчерица, пасынок (Некоторые дети путешествовали только с няней, для них parch = 0)

Датасет состоит из двух частей: train и test, оба в формате .csv. Файл train.csv состоит из 418 записей (пассажиров), test.csv из 891.

4. Программа для обучения модели

Так как модель следует обучить предугадыванию выживания, то возможны лишь два исхода для пассажира: выживет или нет. Правильным решением для такой задачи будет использование методов классификации для обучения модели. Всего будет 2 класса: выживший (1) и невыживший (0). Таким образом, задача является бинарной классификацией.

Для обучения модели была реализована программа на языке Python с использованием библиотек `pandas`, `scikit-learn`, `numpy`, `joblib`.

- `pandas` — для загрузки датасета, обработки пропущенных значений и преобразования признаков;
- `numpy` — для выполнения численных операций и работы с массивами данных;

- `scikit-learn` — для реализации алгоритмов машинного обучения, разбиения данных на обучающую и тестовую выборки, а также оценки качества модели;
- `joblib` — для сохранения и последующей загрузки обученной модели.

На первом этапе выполняется загрузка данных и их предварительная обработка. Из датасета удаляются признаки, не оказывающие существенного влияния на результат классификации (идентификаторы пассажиров, имена и номера билетов). Пропущенные значения возраста заменяются медианным значением, а отсутствующие данные о порте посадки — наиболее часто встречающимся значением.

Категориальные признаки, такие как пол пассажира и порт посадки, преобразуются в числовой формат с использованием бинарного кодирования. Информация о каюте дополнительно обрабатывается: из строкового значения выделяется палуба и числовой номер каюты, что позволяет сохранить полезные пространственные характеристики расположения пассажиров на судне.

В качестве модели классификации был выбран алгоритм Random Forest, основанный на ансамбле решающих деревьев. Данный алгоритм обладает устойчивостью к выбросам, не требует масштабирования признаков и способен эффективно обрабатывать нелинейные зависимости между входными данными.

Обучение модели производится на обучающей выборке с последующим разделением данных на тренировочную и валидационную части. Качество классификации оценивается с использованием метрик точности (Accuracy) и площади под ROC-кривой (AUC). Результаты выполнения

программы представлены на рисунке 1.

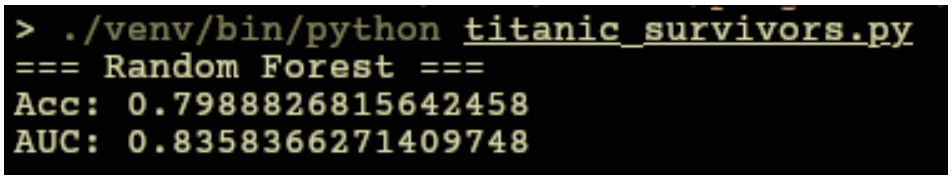


Рисунок 1 — Результат выполнения обученной модели

Как видно из рисунка, показатель точности достаточно высокий ≈ 0.7989 Таким образом, определение класса для большинства объектов выборки прошло успешно. А показатель AUC еще выше ≈ 0.8358, что свидетельствует о высокой способности модели различать пассажиров, выживших и погибших в результате крушения. Как раз та самая точность, на которой будут заметны результаты атак.

5. Анализ уязвимостей модели

Несмотря на достаточно высокие значения метрик качества, модель машинного обучения может обладать рядом уязвимостей, связанных с особенностями обучающего датасета, предобработки данных и используемого алгоритма.

Под уязвимостью в данном случае понимается возможность искусственного изменения входных данных таким образом, чтобы модель выдала неверный результат, либо снижение устойчивости предсказаний при незначительных изменениях признаков.

Зависимость от ключевых признаков.

График самых влиятельных на результат признаков представлен на рисунке 2.

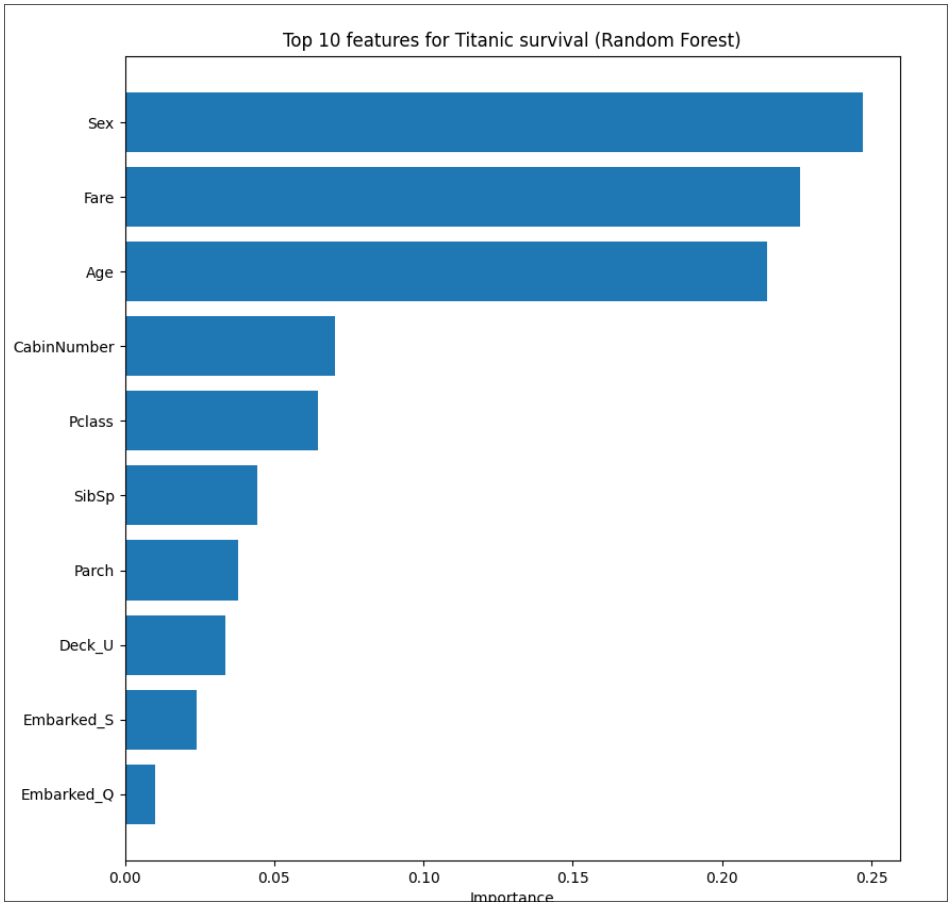


Рисунок 2 — График с самыми влиятельными признаками

В данной модели наибольшее влияние на результат классификации оказывают признаки `Sex`, `Fare` и `Age` с влиянием >= 0.2. `CabinNumber`, `Pclass` и остальные признаки имеют не такое большое влияние на результат <= 0.07.

Это означает, что модель сильно полагается на ограниченное количество признаков.

Отсутствие гладкости предсказаний.

Random Forest формирует решение на основе набора пороговых условий, что приводит к скачкообразному изменению результата при небольших изменениях входных данных. Уязвимостью может быть даже незначительное изменение одного признака (например, возраста на 1-2 года) может изменить путь прохождения по деревьям и привести к

противоположному результату классификации.

6. Атакующие алгоритмы

Теперь, когда уязвимости известны, нужно выбрать атакующие алгоритмы.

Начну с того, зачем вообще нужно проводить атаку на модель.

Системы машинного обучения в своей основе являются строго зависимыми от данных, что влечёт за собой ряд существенных последствий. Любые изменения во входных данных неизбежно отражаются на поведении модели. При этом обучение моделей машинного обучения осуществляется лишь на ограниченном подмножестве данных — тренировочном наборе. На этапе эксплуатации модель начинает взаимодействовать с реальными данными, представляющими генеральную совокупность. Эти данные по своим статистическим характеристикам могут в целом отличаться от тех, которые использовались при обучении и тестировании модели. График аппроксимации функции при наличии подменных данных представлен на рисунке 3.

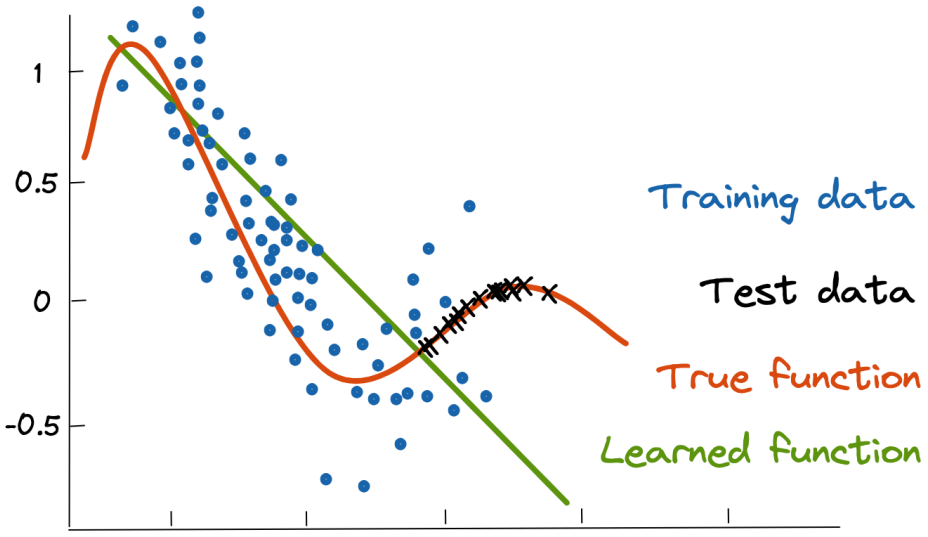


Рисунок 3 — График аппроксимации функции

Модель стремится аппроксимировать неизвестную зависимость, изображённую на графике красной линией. Синим цветом на рисунке обозначены данные тренировочного набора.

Данные тренировочного набора хорошо обобщаются моделью, и аппроксимирующая зелёная прямая демонстрирует высокие значения метрик до тех пор, пока реальные данные сохраняют сходные характеристики с обучающим набором. Однако при изменении свойств входных данных величина потерь начинает возрастать. Таким образом, изменение распределения данных напрямую приводит к изменению поведения модели. При этом такие изменённые данные остаются полностью легитимными для модели и не выходят за рамки допустимого входного пространства.

Следовательно, в процессе эксплуатации модель машинного обучения может демонстрировать значения метрик, существенно отличающиеся от достигнутых на этапе тестирования. На рисунке 3 показан наиболее простой вид расхождения между реальными и тренировочными данными — ковариантный сдвиг. Вместе с тем наибольшую опасность для корректной работы модели представляет так называемый сдвиг концепции, заключающийся в изменении зависимости между входными и выходными переменными.

Дополнительно возникает вопрос о наличии противоборствующей стороны, которая целенаправленно подбирает такие входные данные, способные негативно повлиять на работу модели. Разница между аппроксимирующей прямой и истинной функцией на рисунке 3 соответствует величине потерь. Поведение функции потерь стандартным образом определяется её производной по входным данным. Для многомерного пространства признаков такими характеристиками являются частные производные, а их совокупность образует якобиан. На рисунке 3 пред-

ставлена иллюстрация атак уклонения, основанных на целенаправленных модификациях входных данных, которые приводят к изменению результата классификации после добавления к корректно распознаваемому изображению шума, вычисленного с использованием якобиана функции потерь.

Следует отметить, что подобные модификации входных данных тесно связаны с проблемой устойчивости моделей машинного обучения. Как правило, алгоритм, в котором ошибка, присутствующая во входных данных или возникающая в процессе вычислений, не возрастает или увеличивается незначительно на каждом шаге, считается устойчивым. В противном случае, когда погрешность существенно накапливается от шага к шагу, алгоритм классифицируется как неустойчивый. Таким образом, устойчивость алгоритма представляет собой количественную меру его чувствительности к изменениям исходных данных. На рисунке 4 приведён пример того, как незначительное изменение входных данных может приводить к существенному снижению результативности модели.

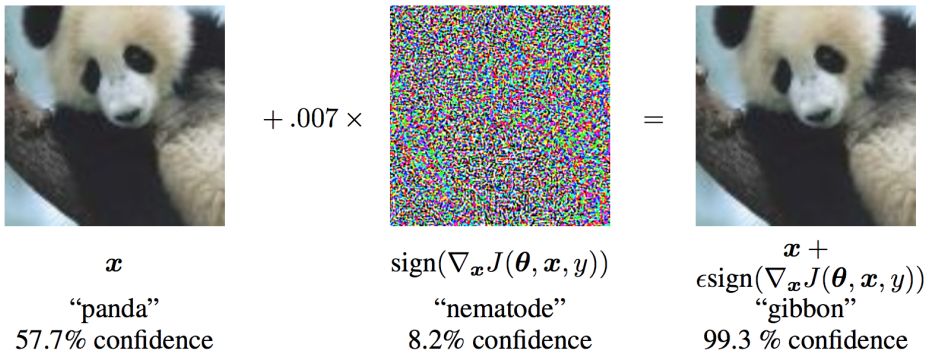


Рисунок 4 — Враждебная атака с применением шума

Вот только есть один нюанс. Как упоминалось ранее, модель использует алгоритм Random Forest. Выражение для предсказания одного дерева в ансамбле Random Forest представлено в формуле (1).

$$f_{\mathrm{DT}}(x) = \sum_k c_k * 1_{x \in R_k} \tag{1}$$

где: $R_k \subset \mathbb{R}^d$ — области признакового пространства, соответствующие листьям дерева,

$c_k \in [0, 1]$ — значение предсказания в листе,

$1_{x \in R_k}$ — индикаторная функция, показывающая, что x попадает в лист R_k.

Данная функция недифференцируема. Для иллюстрации была использована выборка случайных чисел с дополнительным ограничением $y = x > 30$. График функции приведён на рисунке 5.

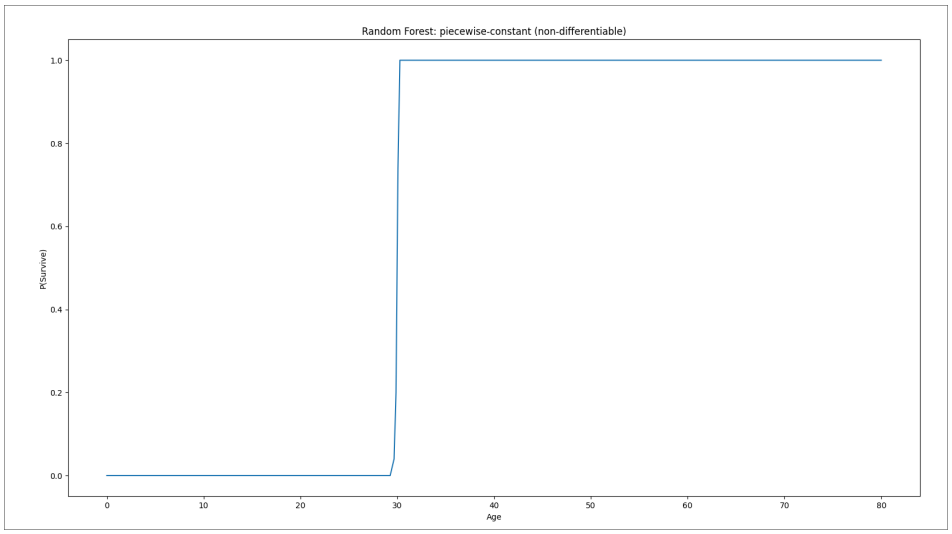


Рисунок 5 — Предсказание одного дерева Random Forest для $x > 30$

Поскольку модели на основе деревьев решений недифференцируемы, прямое вычисление якобиана функции потерь для них невозможно. В связи с этим используется surrogate-модель на основе нейронной сети, аппроксимирующая границу решений исходного классификатора. В отличие от Random Forest, нейронная сеть задаёт гладкую и дифференцируемую функцию, что позволяет вычислять производные функции потерь по

входным признакам и анализировать чувствительность модели к малым возмущениям входных данных.

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(f_{\text{RF}}(x_i) - \hat{f}_{\text{NN}}(x_i; \theta) \right)^2 \tag{2}$$

где: $f_{\text{RF}}(x)$ — предсказание Random Forest,

$\hat{f}_{\text{NN}}(x; \theta)$ — surrogate нейросеть с параметрами θ,

$\hat{f}_{\text{NN}}(x; \theta)$ — градиент surrogate, используемый для оптимизации,

График функции surrogate-модели нейронной сети представлен на рисунке 6.

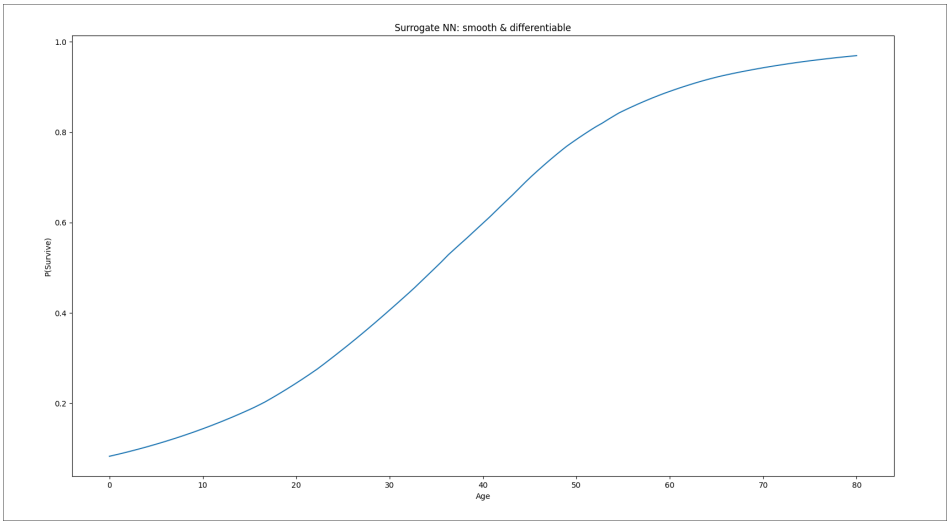


Рисунок 6 — Предсказание surrogate-модели на основе нейронной сети

Такое поведение функции можно объяснить тем, что Surrogate NN использует функцию активации. Функции активации привносят нелинейность в сеть, позволяя ей моделировать сложные структуры данных. конкретно в моем примере, это функция $ReLU$ (3). А в качестве функции затухания используется $Sigmoid$ (4)

$$ReLU(x) = max(0, x) \tag{3}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{4}$$

Теперь для выявления характеристик, чувствительность которых основана на якобиане, напишу программу (см. Приложение 3). Результатом данной программы будет список характеристики с высокой чувствительностью по якобиану для рассматриваемой модели Random Forest. Выполненная программа представлена на рисунке 7.

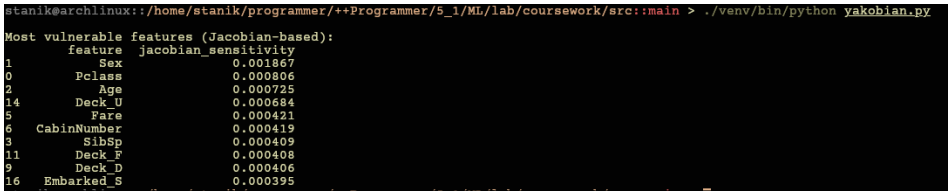


Рисунок 7 — Характеристики с высокой чувствительностью по якобиану

В качестве атакующего алгоритма будет использован Adversarial (attack).

Классификация атак уклонением. При атаке узлоумышленник не может влять ни на обучающее данные (атака отравлением), ни каким-либо образом изменять параметры модели. Злоумышленник может лишь иметь доступ к инофрмации об архитектуре и веса модели или про её поведение на различных входах. Можно декомпозировать модель угроз на следующие аспекты: знания злоумышленника о модели, возможность напрямую конртолировать вход модели, цель злоумышленника, итеративная мощность атаки.

Классификаций атак есть множество. Разберу классификации на основе знаний злоумышленника о модели.

Атаки в режиме белого ящика (white-box). В этой модели угроз злоумышленник обладает значительной информацией о внутреннем устрйостве модели. Чаще всего предполагается знание архитектуры модели и

значения всех её параметров. Это самая сильная модель уггроз.

Чаще всего информация об архмтектуре и параметрах модели используется для вычисление градиента выхода моедли по её входу.

Атаки в режиме черного ящика (black-box). В таких атаках не предполагается детального знания об архитектуре и параметрах модели. Однако предполагается взаимодействие с моделью и возможность наблюдать её выход в зависиомтси от посылаемого ей входа. Можно разделить на 3 вида:

- Атаки на основе выходного распределения модели. Имеются в виду предсказанные моделью вероятности или логиты. Во многих случаях суть данных атак сводится к численной оценке градиента, так как прямое его вычисление невозможно при отсутствии доступа к параметрам модели. В других случаях используются оптимизированные алгоритмы, не требующие градиента, такие как эволюционные алгоритмы или случайный поиск.

- Атаки на основе решения модели. Предполагается наличие доступа к предсказанной моделью метке. Чаще всего такие атаки основываются на лучайной инициализации и постепенном приближении к оригинальному изображению.

- Атаки на основе переносимости. Состязательные примеры могут быть успешно перенесены с одной модели на другую. В этой модели угроз предполагается наличие доступа или к полному датасету, на котором обучалась модель, или к его части для обучения модели-суррогата, для которой и будут синтезироваться состязательные примеры, а затем переноситься н атакуемую модель.

В этой работе рассмотрим атаку на основе переноса.

Атака будет осуществляться следующим образом. Имея доступ к входным данным (x_{train}, x_{test}, y_{train}, y_{test}) и к выходам целевой модели, обучается суррогатная модель, аппроксимирующая поведение целевого классификатора. Далее состязательные примеры синтезируются для суррогатной модели и переносятся на целевую модель за счёт свойства переносимости состязательных примеров.

Обучение суррогатной модели основано на анализе направлений, в которых выходы модели изменяются наиболее существенно. Эти направления определяются с помощью якобиана суррогатной модели $J_f = [\partial f_i(x)/\partial x_j]_{ij}$.

Обучение суррогата проводится итеративно. На каждой эпохе текущая выборка входных данных отправляется на разметку целевой модели, после чего суррогатная модель дообучается на полученных выходных распределениях.

Благодаря переносимости состязательных примеров между различными классами моделей (например, между нейронными сетями и линейными классификаторами), предложенная атака может быть реализована даже при отсутствии информации о внутреннем устройстве целевой модели.

Атаки уклонением (Adversarial attacks) - это разновидность умышленного воздействия на модель, при котором происходит изменение данных с целью получения неверного результата.

Для атаки создается состязательный набор для действующей системы, при работе с которым модель будет функционировать некорректно. Целью подобных атак является получение неверной классификации или идентификации каких-либо объектов.

Чаще всего данным атакам подвергаются сети в системах компью-

терного зрения, чтобы запутать модель в процессе распознавания или определния объектов посредством наложения небольшого шума, неуловимого человеком.

Этот вид атак облаадет особенностями, делающими уязвимыми множество систем, в которых используются алгоритмы машинного обучения. Во-первых, состязательныме примеры могут быть сгенерированы для большинства датасетов, следовательно, атака может быть совершена на модель вне зависимости от её архитектуры.

Во-вторых, состязательные данные могут быть найдены в физическом мире, к примеру, при некачественной фотосъемке объекта, который необходимо распознать или классифицировать.

Пример работы программы представлен на рисунках 8 и 9.

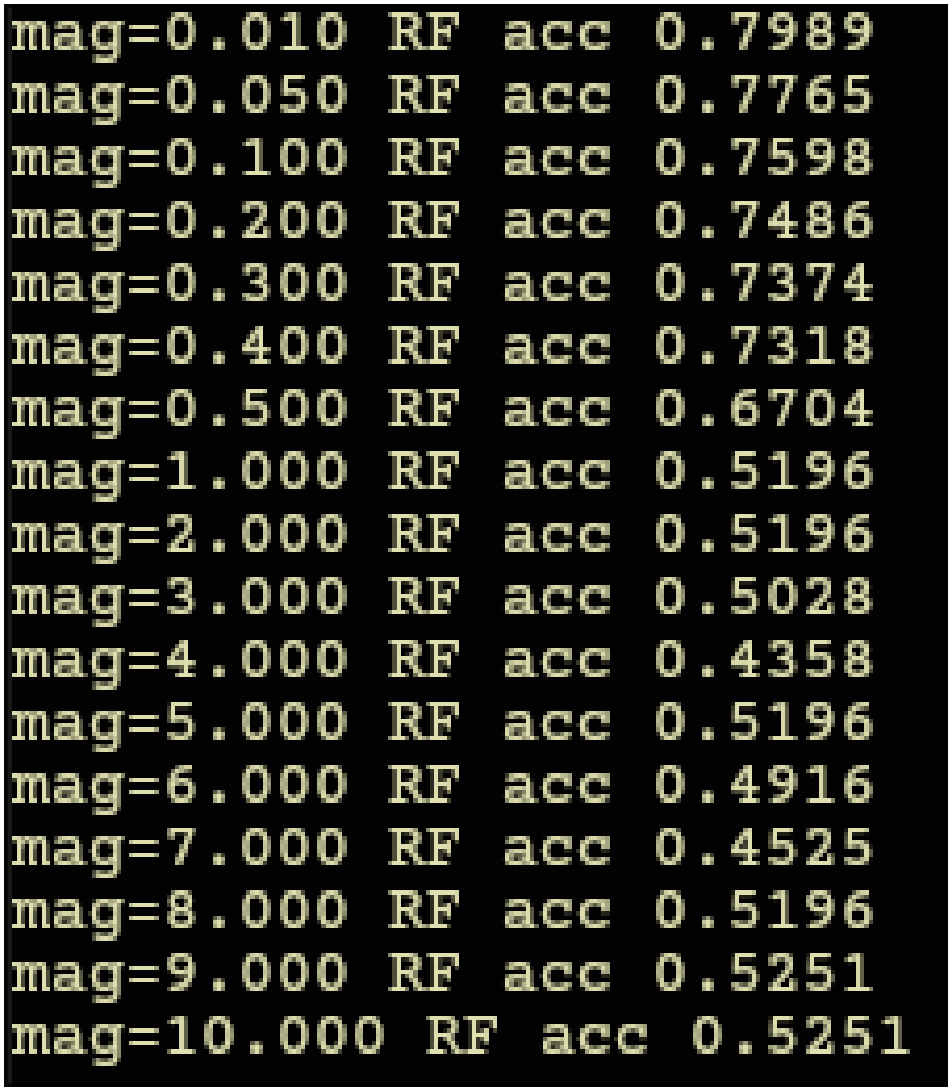


Рисунок 8 — Точность модели после совершения атаки при различных ϵ

	magnitude	Sex_orig	Sex_adv	Sex_delta	Age_orig	Age_adv	Age_delta	Pclass_orig	Pclass_adv	Pclass_delta
0	0.010	0	0	0	24.000	24.000	0.000	3	3	0
1	0.050	0	0	0	24.000	22.000	-2.000	3	3	0
2	0.100	0	0	0	24.000	27.000	3.000	3	3	0
3	0.200	0	0	0	24.000	28.000	4.000	3	3	0
4	0.300	0	0	0	24.000	46.000	22.000	3	3	0
5	0.400	0	0	0	24.000	18.000	-6.000	3	3	0
6	0.500	0	0	0	24.000	17.000	-7.000	3	3	0
7	1.000	0	0	0	24.000	24.000	0.000	3	2	-1
8	2.000	0	0	0	24.000	0.420	-23.580	3	1	-2
9	3.000	0	1	1	24.000	74.000	50.000	3	3	0
10	4.000	0	0	0	24.000	80.000	56.000	3	3	0
11	5.000	0	1	1	24.000	0.420	-23.580	3	3	0
12	6.000	0	1	1	24.000	80.000	56.000	3	1	-2
13	7.000	0	0	0	24.000	0.420	-23.580	3	1	-2
14	8.000	0	1	1	24.000	0.420	-23.580	3	1	-2
15	9.000	0	0	0	24.000	0.420	-23.580	3	1	-2
16	10.000	0	1	1	24.000	80.000	56.000	3	1	-2

Рисунок 9 — Результат выполнения атаки (изменение данных первого пассажира)

Рассмотрим пример подмены характеристик первого пассажира из валидационной выборки. Изначально объект имеет следующие значения: возраст 24 года, класс обслуживания — 3, пол — мужской.

При малых значениях параметра возмущения ϵ_{mag} (0.01–0.1) атака действует крайне аккуратно. Изменения либо отсутствуют, либо сводятся к незначительным целочисленным сдвигам возраста на 1–3 года, что укладывается в естественную вариативность данных и не нарушает семантику объекта. Предсказание модели при этом, как правило, сохраняется.

По мере увеличения ϵ_{mag} до диапазона 0.2–0.5 изменения становятся заметно более агрессивными: возраст может увеличиваться или уменьшаться на десятки лет, что уже оказывает существенное влияние на решение классификатора, несмотря на формальную допустимость значений.

При больших значениях ϵ_{mag} (1.0 и выше) наблюдается резкий рост амплитуды подмены признаков. Возраст пассажира может принимать крайние значения диапазона (0 или 80 лет), а класс обслуживания изменяется с третьего на первый. Такие модификации приводят к смене решения модели и демонстрируют, что для успешного уклонения достаточно манипулировать небольшим числом наиболее чувствительных характеристик.

Начиная с ϵ_{mag} равным 3.0, начинается меняться даже пол человека. Такая атака уже врядли останется незамеченной, но безусловно, как видно из рисунка, модель начинает ошибаться крайне часто. Я напоминаю, что модель должна определить один из двух исходов, что означает что при результат в 50%, модель начинает выбирать решение буквально наугад.

Таким образом, атака имеет двухфазный характер: на малых ϵ_{mag} она действует скрытно и почти незаметно, а при увеличении параметра возмущения переходит в агрессивный режим, быстро разрушая корректность предсказаний модели.

7. Выводы

8. Список источников

1. Намиот Д.Е. Схемы атак на модели машинного обучения [Электронный ресурс] / Режим доступа к данным: https://cyberleninka.ru/article/n/shemy-atak-na-modeli-mashinnogo-obucheniya/viewer, свободный — 14.12.2025.

2. Костюмов В.В Обзор и систематизация атак уклонением на модели компьютерного зрения [Электронный ресурс] / Режим доступа к данным: https://cyberleninka.ru/article/n/obzor-i-sistematizatsiya-atak-ukloneniem-na-modeli-kompyuternogo-zreniya/viewer, свободный — 14.12.2025.

3. Titanic: Machine Learning from Disaster [Электронный ресурс] / Режим доступа к данным: https://www.kaggle.com/competitions/titanic/code, свободный — 14.12.2025.

4. Feedforward Neural Network [Электронный ресурс] / Режим доступа к данным: https://www.geeksforgeeks.org/deep-learning/feedforward-neural-network/, свободный — 14.12.2025.

Приложение 1. Код программы data_preprocessing.py

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
def load_and_preprocess_data(data_dir="titanic/"):
    """
    Load and preprocess Titanic data.
    Parameters:
    ----------
    data_dir : str
        Directory containing the data file
    Returns:
    --------
    tuple : (X_train, X_val, y_train, y_val, df)
    """
    # ---------------- Load data ----------------
    df = pd.read_csv(data_dir + "train.csv")
    # ---------------- Cabin processing ----------------
    df["Cabin"] = df["Cabin"].fillna("Unknown")
    # Deck = первая буква
    df["Deck"] = df["Cabin"].str[0]
    # Числовая часть каюты
    df["CabinNumber"] = (
        df["Cabin"].str.extract(r"(\d+)", expand=False).fillna(-1).astype(int)
    )
    df = pd.get_dummies(df, columns=["Deck"], drop_first=True)
    df = df.drop(columns=["Cabin"])
    # ---------------- Basic cleaning ----------------
    df = df.drop(columns=["PassengerId", "Name", "Ticket"])
    df["Age"] = df["Age"].fillna(df["Age"].median())
    df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
    df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
    df = pd.get_dummies(df, columns=["Embarked"], drop_first=True)
    return df
def split_data(df, test_size=0.2, random_state=42):
    """
    Split data into training and validation sets.
    Parameters:
    ----------
    df : pandas.DataFrame
        Preprocessed dataframe with 'Survived' column
    test_size : float
        Proportion of data for validation set
    random_state : int
        Random seed for reproducibility
    Returns:
    tuple : (X_train, X_val, y_train, y_val)
    """
    # ---------------- Feature/target split ----------------
    X = df.drop(columns=["Survived"])
    y = df["Survived"]
    # ---------------- Train/val split ----------------
    X_train, X_val, y_train, y_val = train_test_split(
        X, y, test_size=test_size, stratify=y, random_state=random_state
    )
    return X_train, X_val, y_train, y_val
def get_processed_data(data_dir="titanic/", test_size=0.2, random_state=42):
    # ---------------- Load data ----------------
    df = pd.read_csv(data_dir + "train.csv")
    # ---------------- Basic sanity filtering ----------------
    # Age: realistic bounds
    df = df[(df["Age"].isna()) | ((df["Age"] >= 0) & (df["Age"] <= 100))]
    # Sex: only valid categories
    df = df[df["Sex"].isin(["male", "female"])]
    # Fare: remove extreme outliers
    df = df[(df["Fare"] >= 0) & (df["Fare"] <= 5000)]
    # Pclass: only valid classes
```

```
    df = df[df["Pclass"].isin([1, 2, 3])]
    # ---------------- Cabin processing ----------------
    df["Cabin"] = df["Cabin"].fillna("Unknown")
    df["Deck"] = df["Cabin"].str[0]
    df["CabinNumber"] = (
        df["Cabin"]
        .str.extract(r"(\d+)", expand=False)
        .fillna(-1)
        .astype(int)
    )
    df = pd.get_dummies(df, columns=["Deck"], drop_first=True)
    df = df.drop(columns=["Cabin"])
    # ---------------- Drop useless columns ----------------
    df = df.drop(columns=["PassengerId", "Name", "Ticket"])
    # ---------------- Missing values ----------------
    df["Age"] = df["Age"].fillna(df["Age"].median())
    df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
    # ---------------- Encoding ----------------
    df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
    df = pd.get_dummies(df, columns=["Embarked"], drop_first=True)
    # ---------------- Split ----------------
    X = df.drop(columns=["Survived"])
    y = df["Survived"]
    X_train, X_val, y_train, y_val = split_data(df)
    # ---------------- Scaling ----------------
    scaler = StandardScaler()
    return X_train, X_val, y_train, y_val, df, scaler
def load_models_from_file(data_dir="models/"):
    scaler = joblib.load(data_dir + "scaler.pkl")
    dt = joblib.load(data_dir + "decision_tree.pkl")
    gnb = joblib.load(data_dir + "gaussian_nb.pkl")
    logreg = joblib.load(data_dir + "logreg.pkl")
    rf = joblib.load(data_dir + "random_forest.pkl")
    svm = joblib.load(data_dir + "svm.pkl")
    return scaler, dt, gnb, logreg, rf, svm
```

Приложение 2. Код программы random_forest.py

```
import pandas as pd
import numpy as np
import os
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from data_preprocessing import get_processed_data
# ------------------------------------------------------------
# Prepare output dir
# ------------------------------------------------------------
os.makedirs("models", exist_ok=True)
# ------------------------------------------------------------
# Load data
# ------------------------------------------------------------
X_train, X_val, y_train, y_val, df, scaler = get_processed_data(
    data_dir="titanic/", test_size=0.2, random_state=42
)
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
# Save scaler
joblib.dump(scaler, "models/scaler.pkl")
```

```
# ============================================================
# Random Forest
# ============================================================
rf = RandomForestClassifier(n_estimators=400, random_state=42)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_val)
proba_rf = rf.predict_proba(X_val)[:, 1]
print("=== Random Forest ===")
print("Acc:", accuracy_score(y_val, pred_rf))
print("AUC:", roc_auc_score(y_val, proba_rf))
print()
joblib.dump(rf, "models/random_forest.pkl")
```

Приложение 3. Код программы diff.py

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
# ----- Generate simple 1D dataset (Age only) -----
np.random.seed(42)
X = np.linspace(0, 80, 400).reshape(-1, 1)
y = (X[:, 0] > 30).astype(int)  # simple threshold rule
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# ----- Train Random Forest -----
rf = RandomForestClassifier(n_estimators=50, max_depth=3, random_state=42)
rf.fit(X, y)
rf_pred = rf.predict_proba(X)[:, 1]
# ----- Train Surrogate NN -----
class Surrogate(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x)
model = Surrogate()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.BCELoss()
X_t = torch.tensor(X_scaled, dtype=torch.float32)
y_t = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)
for _ in range(200):
    optimizer.zero_grad()
    out = model(X_t)
    loss = loss_fn(out, y_t)
    loss.backward()
    optimizer.step()
with torch.no_grad():
    nn_pred = model(X_t).numpy().ravel()
# ----- Plot Random Forest -----
plt.figure()
plt.plot(X[:, 0], rf_pred)
plt.xlabel("Age")
plt.ylabel("P(Survive)")
plt.title("Random Forest: piecewise-constant (non-differentiable)")
plt.show()
# ----- Plot Surrogate NN -----
```

```
plt.figure()
plt.plot(X[:, 0], nn_pred)
plt.xlabel("Age")
plt.ylabel("P(Survive)")
plt.title("Surrogate NN: smooth & differentiable")
plt.show()
```

Приложение 4. Код программы yakobian.py

```
import torch
import torch.nn as nn
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score
from data_preprocessing import get_processed_data
# ---------------- Setup ----------------
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
X_train, X_val, y_train, y_val, df, scaler = get_processed_data()
feature_names = list(X_train.columns)
# scale for surrogate
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
y_train_t = torch.tensor(y_train.values, dtype=torch.long).to(device)
X_val_t = torch.tensor(X_val_scaled, dtype=torch.float32).to(device)
y_val_t = torch.tensor(y_val.values, dtype=torch.long).to(device)
seed = 42
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
# ---------------- Surrogate NN ----------------
class SurrogateMLP(nn.Module):
    def __init__(self, n):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 2),
        )
    def forward(self, x):
        return self.net(x)
model = SurrogateMLP(X_train.shape[1]).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
# ---------------- Train ----------------
for epoch in range(75):
    optimizer.zero_grad()
    out = model(X_train_t)
    loss = criterion(out, y_train_t)
    loss.backward()
    optimizer.step()
# ---------------- Jacobian-based sensitivity ----------------
model.eval()
X_val_t.requires_grad = True
logits = model(X_val_t)
loss = criterion(logits, y_val_t)
loss.backward()
jacobian_abs = X_val_t.grad.abs().cpu().numpy()
# mean sensitivity per feature
feature_sensitivity = jacobian_abs.mean(axis=0)
sensitivity_df = pd.DataFrame({
    "feature": feature_names,
    "jacobian_sensitivity": feature_sensitivity
```

```
}).sort_values(by="jacobian_sensitivity", ascending=False)
print("\nMost vulnerable features (Jacobian-based):")
print(sensitivity_df.head(10))
```

Приложение 5. Код программы adversarial_attack.py

```
from data_preprocessing import get_processed_data
from data_preprocessing import load_models_from_file
import torch
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, roc_auc_score
seed = 42
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
X_train, X_val, y_train, y_val, df, scaler = get_processed_data(
    test_size=0.2, random_state=seed
)

scaler, dt, gnb, logreg, rf, svm = load_models_from_file()
feature_names = list(X_train.columns)
allowed_features = ["Sex", "Age", "Pclass"]
allowed_idx = [feature_names.index(f) for f in allowed_features if f in feature_names]
onehot_groups = []
# Simple heuristic: columns that start with same prefix before '_' and are categorical dummies
prefixes = {}
for col in feature_names:
    if "_" in col:
        p = col.split("_")[0]
        prefixes.setdefault(p, []).append(col)
for p, cols in prefixes.items():
    if len(cols) > 1:
        onehot_groups.append(cols)
# filter out numeric groups accidentally included (keep likely categorical groups)
# e.g., Deck_, Embarked_
onehot_groups = [
    g for g in onehot_groups if any(c.startswith(("Deck", "Embarked")) for c in g)
]
print("One-hot groups:", onehot_groups)
# bounds for clipping in original space
X_all = pd.concat([X_train, X_val], axis=0)
orig_min = X_all.min().values
orig_max = X_all.max().values
# for rounding/validating one-hot: function that sets max index to 1 others to 0
def project_onehot_row(row_arr, group_cols, feature_names):
    arr = row_arr.copy()
    for group in group_cols:
        idxs = [feature_names.index(c) for c in group]
        if max(arr[idxs]) <= 0:
            # nothing positive -> keep original argmax (to avoid all zeros)
            # set the max among original (use current values)
            arg = int(np.argmax(arr[idxs]))
        else:
            arg = int(np.argmax(arr[idxs]))
        for j, idx in enumerate(idxs):
            arr[idx] = 1.0 if j == arg else 0.0
    return arr
# ---------------- Evasion attack generation ----------------
def evasion_attack(X_orig, allowed_idx, magnitude=0.05):
    """
    Simple evasion attack: add random perturbations to allowed features within given magnitude.
    X_orig: np.array (n_samples, n_features) in original (unscaled) space
    allowed_idx: list of feature indices allowed to perturb
    magnitude: fraction of the feature range to perturb
    """
    X_adv = X_orig.copy()
    for idx in allowed_idx:
```

```
        # compute feature range
        f_min = X_all.iloc[:, idx].min()
        f_max = X_all.iloc[:, idx].max()
        f_range = f_max - f_min
        # random perturbation in [-magnitude*range, +magnitude*range]
        perturb = (np.random.rand(X_orig.shape[0]) * 2 - 1) * magnitude * f_range
        X_adv[:, idx] += perturb
    # clip to original bounds
    # project one-hot groups row-by-row
    for i in range(X_adv.shape[0]):
        X_adv[i] = project_onehot_row(X_adv[i], onehot_groups, feature_names)
        X_adv[i] = project_semantic_constraints(X_adv[i], feature_names)
    # clip to original observed bounds
    X_adv = np.minimum(np.maximum(X_adv, orig_min), orig_max)
    return X_adv
def project_semantic_constraints(row_arr, feature_names):
    arr = row_arr.copy()
    # Age: uint >= 0
    if "Age" in feature_names:
        i = feature_names.index("Age")
        arr[i] = int(round(arr[i]))
        arr[i] = max(arr[i], 0)
    # Pclass: uint in {1,2,3}
    if "Pclass" in feature_names:
        i = feature_names.index("Pclass")
        arr[i] = int(round(arr[i]))
        arr[i] = min(max(arr[i], 1), 3)
    # Parch: uint in [0,10]
    if "Parch" in feature_names:
        i = feature_names.index("Parch")
        arr[i] = int(round(arr[i]))
        arr[i] = min(max(arr[i], 0), 100)
    # Sex: uint in [0,1]
    if "Sex" in feature_names:
        i = feature_names.index("Sex")
        arr[i] = int(round(arr[i]))
        arr[i] = min(max(arr[i], 0), 1)
    return arr
# Run evasion attack
eps_magnitude_list = [0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ]  # fractio
results = []
X_val_np = X_val.values  # original unscaled

for mag in eps_magnitude_list:
    X_adv = evasion_attack(X_val_np, allowed_idx, magnitude=mag)
    X_adv_df = pd.DataFrame(X_adv, columns=X_train.columns)
    # RF
    preds_rf_adv = rf.predict(X_adv_df)
    proba_rf_adv = rf.predict_proba(X_adv_df)[:, 1]
    acc_rf = accuracy_score(y_val, preds_rf_adv)
    auc_rf = roc_auc_score(y_val, proba_rf_adv)
    results.append({
        "magnitude": mag,
        "RF_acc": acc_rf,
        "RF_auc": auc_rf,
    })
    print(f"mag={mag:.3f} RF acc {acc_rf:.4f}")
# ====================== TABULAR TRACE FOR SINGLE OBJECT ====================
# prettier output: limit float precision
pd.set_option("display.float_format", "{:.3f}".format)
print("\n=== EVASION TRACE (single sample) ===")
sample_idx = 0  # first validation sample
x_orig = X_val_np[sample_idx].copy()
x_orig_s = pd.Series(x_orig, index=feature_names)
rows = []
for mag in eps_magnitude_list:
    X_adv = evasion_attack(
```

```
            X_val_np[sample_idx:sample_idx+1],
            allowed_idx,
            magnitude=mag
        )
        x_adv = X_adv[0]
        x_adv_s = pd.Series(x_adv, index=feature_names)
        row = {
            "magnitude": mag,
        }
        for f in allowed_features:
            row[f"{f}_orig"] = x_orig_s[f]
            row[f"{f}_adv"] = x_adv_s[f]
            row[f"{f}_delta"] = x_adv_s[f] - x_orig_s[f]
        rows.append(row)
trace_df = pd.DataFrame(rows)
print(trace_df)
```